Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering & Research Ravet, Pune
An Autonomous Institute | NBA Accredited (4 UG Programs) | NAAC A++ Accredited | ISO 21001:2018 Certified
PUNE-412101
Department of

COMPUTER ENGINEERING

LABORATORY MANUAL

PROGRAM	**COMPUTER ENGINEERING**
COURSE	**PCC-205-COM (2025)**
COURSE NAME	**OBJECT ORIENTED PROGRAMMING & COMPUTER GRAPHICS LAB**
CLASS	**SY B.Tech –SEM I**

SY B.Tech of Computer Engineering (2025 Course)

PCC-205-COM: OOP and Computer Graphics Laboratory

Teaching Scheme:	Credit	Examination Scheme:
	01	TW: 25 Marks
		PR: 25 Marks

Prerequisite Courses, if any: BSC-101-ASH Engineering Mathematics I
ESC-101-ASH Programming in C

Companion Course, if any: Object oriented Programming, Computer Graphics

Course Objectives:

The course aims to:

1. Implement object-oriented programs in C++ using classes, constructors, inheritance, Polymorphism and dynamic memory allocation.
2. Develop C++ programs to handle exceptions and perform file operations for persistent data Storage.
3. Implement and test fundamental computer graphics algorithms for drawing, filling, clipping, And transforming graphical objects.
4. Develop a graphics-based application using C++ by integrating object-oriented programming concepts with computer graphics techniques

Course Outcomes:

Upon successful completion of this course, students will be able to:

1. **PCC205.1:** Implement C++ programs using classes, objects, constructors, destructors, and static members for data modeling problems
2. **PCC205.2:** Develop C++ programs using inheritance, function overriding, and runtime polymorphism to solve real-world problems.
3. **PCC205.3:** Implement robust C++ programs using exception handling and file handling techniques for reliable data processing.
4. **PCC205.4:** Implement fundamental computer graphics algorithms for drawing, filling, and clipping graphical primitives using C++.
5. **PCC205.5:** Apply 2-D geometric transformations on graphical objects using C++ programming.

Guidelines for Instructor's Manual

The instructor's manual/Lab Manual is to be developed as a hands-on resource and reference. The instructor's manual need to include prologue (about University/program/ institute/ department/foreword/ preface), curriculum of course, conduction and Assessment guidelines, topics under consideration-concept, objectives, outcomes, set of typical applications/assignments/guidelines, references.

Guidelines for Student's Laboratory Journal

The laboratory assignments are to be submitted by student in the form of journal. Journal consists of prologue, Certificate, table of contents, and handwritten write-up of each assignment (Title, Objectives, Problem Statement, Outcomes, software and Hardware requirements, Date of Completion, Assessment grade/marks and assessor's sign, Theory Concept in brief, algorithm, flowchart, test cases, Test Data Set(if applicable), mathematical model (if applicable), conclusion/analysis. Program codes with sample output of all performed assignments are to be submitted as softcopy. As a conscious effort and little contribution towards Green IT and environment awareness, attaching printed papers as part of write-ups and program listing to journal may be avoided. Students programs maintained on cloud or college server by Laboratory In-charge is highly encouraged. For reference one or two journals may be maintained with program prints at Laboratory for accreditation purposes.

Guidelines for Laboratory /TW Assessment

Continuous assessment of laboratory work should be done based on overall performance and Laboratory assignments performance of students. Each Laboratory assignment assessment should be assigned grade/marks based on parameters with appropriate weightage. Suggested parameters for overall assessment as well as each Laboratory assignment assessment include timely completion performance, innovation, efficient codes, punctuality and neatness.

Guidelines for Practical Examination

Both internal and external examiners should jointly set problem statements. During practical assessment, the expert evaluator should give the maximum weightage to the satisfactory implementation of the problem statement. The supplementary and relevant questions may be asked at the time of evaluation to test the students for advanced learning, understanding of the fundamentals, effective and efficient implementation. So encouraging efforts, transparent evaluation and fair approach of the evaluator will not create any uncertainty or doubt in the minds of the students. So adhering to these principles will consummate our team efforts to the promising start of the student's academics.

Guidelines for Laboratory Conduction

The instructor is expected to frame the assignments by understanding the prerequisites, technological aspects, utility and recent trends related to the topic. The assignment framing policy needs to address the average students and inclusive of an element to attract and promote the intelligent students. The instructor may set multiple sets of assignments and distribute them among batches of students.

It is appreciated if the assignments are based on real world problems/applications. Encourage students for appropriate use of Hungarian notation, proper indentation and comments. Use of open source software is to be encouraged. In addition to these, instructors may assign one real life application in the form of a mini-project based on the concepts learned. Instructors may also set one assignment or mini-project that is suitable to respective branch beyond the scope of the syllabus.

Operating System recommended:- 64-bit Open source Linux or its derivative

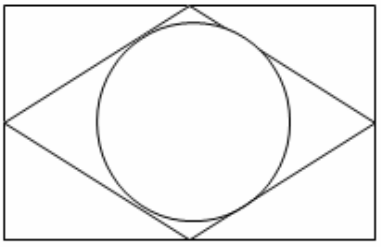
Programming tools recommended: - Open Source Java Open JDK,

Programming IDE like: BlueJ, Eclipse, NetBeans, JDeveloper.

NOTE:** ***Part-A: 5 Assignments, Part- B: 6 Assignments, Part-C (Mini Project): Mandatory Assignment.

Guidelines for Practical Examination

Practical examination gauge students' knowledge and skills based on the spoken word, typically guided by questions or small tasks. A pair of examiners must design appropriate questions for each learning outcome. They should focus on depth rather than breadth. They should include potential follow up questions and prompts based on different types of answers. Examiners should standardize the number of questions, difficulty of questions, and the time allotted. Questions should be based on the practical assignments performed in the term work and not on the entire syllabus.

Suggested List of Laboratory Experiments/Assignments (Implementation of each problem statement is mandatory.)	
Part I : Object Oriented Programming	
Sr.No.	**Group A (All six are compulsory)**
1.	Design and implement a C++ program to create a class Student with data members for roll number, name, and marks. Implement member functions to accept and display the student details
2.	Design a C++ program to implement a class Bank Account that uses constructors to initialize account details and a destructor to indicate object destruction. Use a static data member to maintain the count of total bank accounts created.
3.	Develop a C++ program demonstrating inheritance by creating a base class Employee and derived classes such as Permanent Employee and Contract Employee. Override member functions in derived classes to calculate and display employee salary details.
4.	Write a C++ program to demonstrate runtime polymorphism by creating a base class Shape with a virtual function to calculate area. Implement derived classes like Rectangle and Circle that override the base class function.
5.	Design a C++ program to perform arithmetic division of two numbers. Handle exceptions such as division by zero and invalid input using exception handling mechanisms.
6.	Create a C++ program to store and retrieve records of employees using file handling. Implement a class Employee and perform write and read operations on files using C++ file streams.
	Part II : Computer Graphics
SR.No.	**Group B (Any FOUR)**
1.	Write a C++ Program to draw the following pattern using The DDA line and Bresenham's Circle drawing algorithm. 
2.	Write a **menu-driven C++ program** to draw a **concave polygon** and fill it with a user-defined colour using the **Scan Line polygon filling algorithm**
3.	Write a **C++ program** to implement the **Cohen–Sutherland line clipping algorithm** for clipping a line against a rectangular clipping window.
4.	Write a **C++ program** to implement the **Sutherland–Hodgeman polygon clipping algorithm** to clip a polygon against a rectangular clipping window.

5.	Write a **C++ program** to implement **2-D transformations** such as translation, scaling, rotation, reflection, and shearing on a given object.
6.	Write a **C++ program** to implement **rotation of a 2-D object about an arbitrary point.**

Sr. No.	Group C - Mini Project
	Develop a **graphics-based application in C++** using an **open-source graphics library**, incorporating **object-oriented programming concepts** for design and implementation.

	Learning Resources
1.	a) **Text Books:** 1. E Balaguruswamy, (2023). Programming with JAVA: A Primer. 7th edition. India: McGraw Hill Education. 2. Herbert Schildt, (2021). Java: The complete reference, 13th edition. McGraw-Hill Education. **Reference Books:** 1. Paul Deitel and Harvey Detail, Java: How to Program, Pearson's Publication, 9thEdition 2. Horstmann, C. S. (2023). Core Java - Vol. I – Fundamentals (Vol. 12). Pearson Education 3. Dr. Samit Bhattacharya, Computer Graphics, Oxford University Press, ISBN-13: 978-0-19-809619-1; ISBN-10: 0-19-809619-4. 4. D. Rogers, "Procedural Elements for Computer Graphics", 2ndEdition, Tata McGraw-Hill Publication, 2001, ISBN 0 – 07 – 047371 – 4 MOOC / NPTEL/YouTube Links: - 1. Programming In Java: https://onlinecourses.nptel.ac.in/noc25_cs57/preview 2. https://archive.nptel.ac.in/courses/106/103/106103224/ 3. https://archive.nptel.ac.in/courses/106/102/106102065/ 4. https://nptel.ac.in/courses/106106090

	E-Books : 1. https://www.iitg.ac.in/samit/Computer%20Graphics.pdf 2. https://open.umn.edu/opentextbooks/textbooks/introduction-to-computer-graphics 3. http://www2.cs.uidaho.edu/~jeffery/courses/324/lecture.html

CO-PO Mapping Table:

CO\PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PO11	PSO 1	PSO 2	PSO 3
PCC205.1	3	2	2	-	3	-	-	-	-	-	2	3	3	1
PCC205.2	3	3	3	-	3	-	-	-	-	-	2	3	3	2
PCC205.3	3	3	2	2	3	-	-	-	-	-	2	3	3	2
PCC205.4	3	3	3	2	3	-	-	-	-	-	2	3	2	1
PCC205.5	3	3	3	2	3	-	-	-	-	-	2	3	2	1
AVERAGE:	**3.00**	**2.8**	**2.6**	**1.2**	**3.00**	**-**	**-**	**-**	**-**	**-**	**2.0**	**3.0**	**2.6**	**1.4**

Assignment No. 1

Title	Design and implement a C++ program to create a class Student with data members for roll number, name, and marks. Implement member functions to accept and display the student details
Aim/Problem Statement	Design and implement a C++ program to create a class named Student with data members **roll number**, **name**, and **marks**. Implement member functions to **accept** the student details from the user and **display** the entered details. Create an object of the Student class and demonstrate the functionality by calling the member functions.
CO Mapped	**CO 1**
Pre -requisite	• Basics of C++ programming • Structure of a C++ program • Classes and Objects • Data Members and Member Functions • Access Specifiers (`private` and `public`) • Input and Output using `cin` and `cout` • String data type (`string`) and `getline()` • Creating objects and calling member functions
Learning Objective	1. Understand the concept of **classes and objects** in C++. 2. Create a class with appropriate **data members** and **member functions**. 3. Use **access specifiers** (private and public) to implement data encapsulation. 4. Accept input from the user using cin and getline(). 5. Display object data using member functions. 6. Create objects of a class and invoke member functions to perform operations. 7. Apply the fundamentals of Object-Oriented Programming (OOP) to solve simple real-world problems.

Theory:

A **class** is a user-defined data type in C++ that groups data members and member functions into a single unit. It acts as a blueprint for creating objects.

An **object** is an instance of a class. It occupies memory and is used to access the data members and member functions of the class.

Data Members store the attributes of an object, while **Member Functions** define the operations that can be performed on the object.

In this experiment, the Student class contains:

- Roll Number
- Name
- Marks

The class provides:

- accept() function to read student details.
- display() function to display student details.

Algorithm

1. Start.
2. Define a class Student.
3. Declare data members: rollNo, name, and marks.
4. Define a member function accept() to read student details.
5. Define a member function display() to display student details.
6. Create an object of the Student class in main().
7. Call the accept() function.
8. Call the display() function.
9. Stop.

Flowchart:

(Students Should Draw Flowchart Here)

Conclusion:

This experiment demonstrates the implementation of a simple class in C++. It illustrates how data members and member functions are grouped together in a class, how objects are created, and how member functions are invoked to perform operations on object data.

Questions:

1. Why are data members generally declared as private?
2. What is the difference between a class and an object?
3. What is the difference between private and public access specifiers?
4. How do you create an object of a class?

Assignment No. 2

Title	Design a C++ program to implement a class Bank Account that uses constructors to initialize account details and a destructor to indicate object destruction. Use a static data member to maintain the count of total bank accounts created.
Aim/Problem Statement	Design and implement a C++ program to create a class named BankAccount that stores account details such as **account number**, **account holder name**, and **balance**. Use a **parameterized constructor** to initialize the account details when an object is created and a **destructor** to indicate when an object is destroyed. Implement a **static data member** to maintain and display the total number of bank account objects created. Create multiple objects of the BankAccount class to demonstrate the working of constructors, destructors, and the static data member.
CO Mapped	**CO 2**
Pre -requisite	• Basics of C++ programming • Structure of a C++ program • Classes and Objects • Data Members and Member Functions • Access Specifiers (`private` and `public`) • Constructors (default and parameterized) • Destructors • Static Data Members and Static Member Functions • Object creation and destruction • Input and Output using `cin` and `cout` • Basic understanding of Object-Oriented Programming (OOP) concepts such as encapsulation and object lifecycle

Learning Objective	1. Understand the purpose and use of **constructors** for initializing objects in C++. 2. Understand the role of **destructors** in object destruction and resource cleanup. 3. Create and initialize objects using a **parameterized constructor**. 4. Implement and use **static data members** to maintain information common to all objects of a class. 5. Demonstrate object creation and destruction during program execution. 6. Differentiate between **instance data members** and **static data members**. 7. Apply object-oriented programming concepts such as **encapsulation**, **constructors**, **destructors**, and **static members** in designing a real-world application like a bank account system.

Theory:

Constructor

A constructor is a **special member function** that is automatically invoked whenever an object is created.

Characteristics of Constructors

- Constructor name is the same as the class name.
- It has no return type.
- It is called automatically during object creation.
- It initializes the object's data members.
- Constructors can be overloaded.

Types of Constructors

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor

This experiment uses a **parameterized constructor**.

Parameterized Constructor

A parameterized constructor accepts values during object creation.

Syntax

```
BankAccount(int accNo, string name, float bal)
{
```

```
    accountNo = accNo;
    accountHolder = name;
    balance = bal;
}
```

Advantages

- Initializes objects with user-defined values.
- Reduces the need for separate initialization functions.
- Ensures that every object starts in a valid state.

Destructor

A destructor is a special member function that is automatically called when an object goes out of scope or is explicitly destroyed.

Characteristics

- Destructor name is the class name preceded by ~.
- It has no return type.
- It cannot accept arguments.
- Only one destructor is allowed in a class.
- It cannot be overloaded.

Syntax

```
~BankAccount()
{
    cout << "Object Destroyed";
}
```

Purpose of Destructor

- Indicates the end of an object's lifetime.
- Releases dynamically allocated memory.
- Closes files or database connections.
- Frees other system resources.

In this experiment, the destructor simply displays a message indicating that the object has been destroyed.

Static Data Member

Normally, every object has its own copy of data members. However, a **static data member** is shared among all objects of a class.

Only **one copy** of a static member exists, irrespective of the number of objects created.

Syntax

```
class BankAccount
{
```

```
    static int count;
};
```

Initialization is done outside the class.

```
int BankAccount::count = 0;
```

Features

- Shared by all objects.
- Memory is allocated only once.
- Can be accessed without creating an object.
- Used to maintain common information.

Static Member Function

A static member function can access only static members of the class.

It is invoked using the class name.

Syntax

```
static void displayCount()
{
    cout << count;
}
```

Function call:

```
BankAccount::displayCount();
```

In this experiment:

- The **parameterized constructor** initializes the account number, account holder's name, and balance.
- The **destructor** displays a message when an object is destroyed.
- A **static data member** keeps track of the total number of bank account objects created.

Algorithm:

1. Start.
2. Define a class BankAccount.
3. Declare private data members:

- Account Number
- Account Holder Name
- Balance

4. Declare a static data member count.
5. Define a parameterized constructor to initialize account details and increment count.
6. Define a member function display() to display account details.

7. Define a static member function displayCount() to display the total number of accounts created.
8. Define a destructor to display an object destruction message.
9. In main(), create multiple BankAccount objects.
10. Display account details.
11. Display the total number of accounts created.
12. End the program.

Flowchart: (Students should Draw Flowchart Here)

Conclusion

This experiment demonstrates the use of **parameterized constructors** for object initialization, **destructors** for indicating object destruction, and **static data members** for maintaining information common to all objects of a class. It also illustrates the object lifecycle and reinforces key Object-Oriented Programming concepts in C++.

Questions

1. What is a constructor? Why is it used?
2. What is a destructor? When is it called?
3. Why is a static data member shared among all objects?
4. How is a static data member initialized?

Assignment No. 3

Title	Develop a C++ program demonstrating inheritance by creating a base class Employee and derived classes such as Permanent Employee and Contract Employee. Override member functions in derived classes to calculate and display employee salary details.
Aim/Problem Statement	Design and implement a C++ program to demonstrate **inheritance** by creating a base class named Employee and derived classes such as PermanentEmployee and ContractEmployee. The base class should contain common employee details such as **employee ID**, **employee name**, and **basic salary**. The derived classes should override member functions to calculate and display the salary based on their respective pay structures. Create objects of the derived classes and demonstrate inheritance, function overriding, and code reusability by displaying the salary details of different types of employees.
CO Mapped	**CO 2**
Pre-requisite	Basics of C++ programming Structure of a C++ program Classes and Objects Data Members and Member Functions Access Specifiers (`private`, `protected`, and `public`) Constructors and Destructors Function Overriding Inheritance (Single Inheritance) Base Class and Derived Class concepts Use of the `protected` access specifier in inheritance Object creation and member function invocation Input and Output using `cin` and `cout` Fundamentals of Object-Oriented Programming (OOP), including: 1. Encapsulation 2. Inheritance 3. Polymorphism 4. Code Reusability
Learning Objective	1. Understand the concept of **inheritance** and its role in code reusability. 2. Create a **base class** and **derived classes** using single inheritance in C++. 3. Differentiate between the responsibilities of a base class and derived classes. 4. Implement **function overriding** to provide specialized behavior in derived classes. 5. Use the **`protected` access specifier** to enable controlled access to inherited data members. 6. Calculate and display salary details for different types of employees using derived class implementations. 7. Apply Object-Oriented Programming (OOP) concepts such as **inheritance**, **polymorphism (function overriding)**, **encapsulation**, and **code reusability**

	in designing a real-world employee management application.

Theory:

What is Inheritance?

Inheritance is one of the most important features of Object-Oriented Programming (OOP). It allows a new class (derived class) to acquire the properties and behaviors of an existing class (base class). Inheritance promotes **code reusability**, **extensibility**, and **maintainability** by allowing programmers to build new classes based on existing ones.

Example: In the following example, Animal is the base class and Dog, Cat and Cow are derived classes that extend the Animal class.

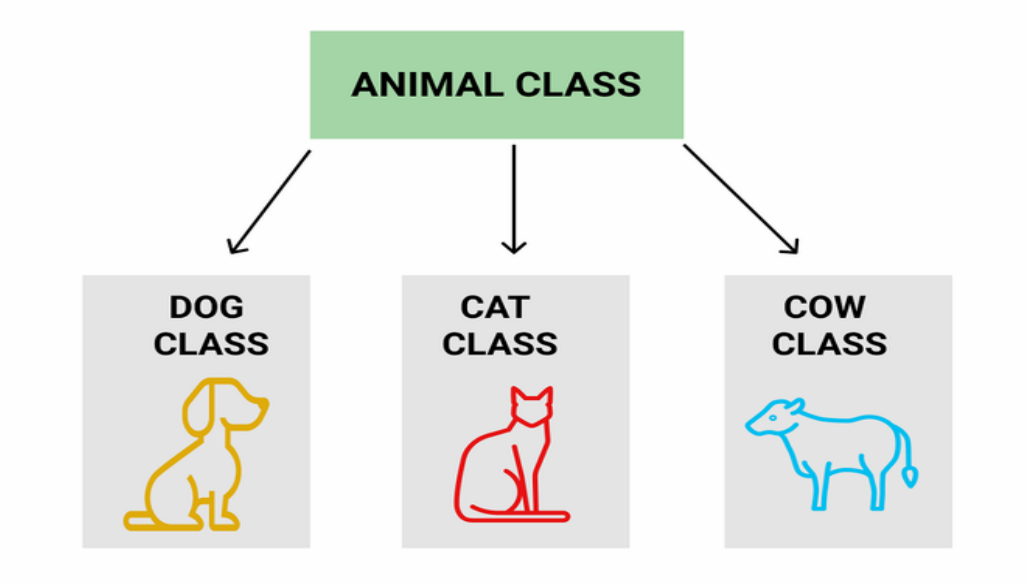


Figure: Base class derived to three subclasses

In OOP, it is possible to inherit attributes and methods from one class to another. We group the "inheritance concept" into two categories:

- **subclass** (child) - the class that inherits from another class
- **superclass** (parent) - the class being inherited from

Subclass

- A subclass, also known as **child class** or **derived class**, is the class that inherits the properties and behaviors of another class. So, if A and B are two classes and if B class inherits A class, then the B class is called the subclass **Superclass**

- A super class, also known as **parent class** or **base class**, is the class whose properties and behaviors are inherited by the subclass. So, if A and B are two classes and if B class inherits A class, then A class is called the superclass.

Types of Inheritance:

Below are the different types of inheritance which are supported by Java.

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance
- Hybrid Inheritance

In this experiment, the Employee class acts as the **base class**, while PermanentEmployee and ContractEmployee act as **derived classes**. Both derived classes inherit the common employee details from the base class and implement their own salary calculation methods.

Function Overriding

Function overriding allows a derived class to redefine a member function already defined in the base class with the same name and parameters.

It enables different classes to provide their own implementation of the same function.

Example:

```
void calculateSalary()
{
   // Salary calculation
}
```

Each derived class provides its own implementation.

Access Specifiers

Private

- Accessible only within the same class.

Protected

- Accessible within the class and its derived classes.

Public

- Accessible from anywhere in the program.

In inheritance, protected members are commonly used because they can be directly accessed by derived classes.

Advantages of Inheritance

- Promotes code reusability.
- Reduces code duplication.
- Improves maintainability.
- Supports software extensibility.

Algorithm

1. Start.
2. Define a base class Employee.
3. Declare employee ID, employee name, and basic salary.
4. Define member functions to accept and display employee details.
5. Create derived class PermanentEmployee.
6. Override the salary calculation function.
7. Create derived class ContractEmployee.
8. Override the salary calculation function.
9. Create objects of both derived classes.
10. Accept employee details.
11. Calculate salary.
12. Display employee details and salary.
13. Stop.

Flowchart: (Students Should Draw Flowchart Here)

Conclusion:
This experiment demonstrates the implementation of **single inheritance** in C++. It illustrates how derived classes inherit common properties from a base class while providing their own implementation of the salary calculation function through **function overriding**. The program highlights the benefits of inheritance, such as **code reusability**, **extensibility**, and **runtime polymorphism**, making it suitable for modeling real-world applications like employee management systems.

Questions:

1. What is the difference between a base class and a derived class?
2. What is the purpose of the `virtual` keyword?
3. What is runtime polymorphism?
4. What is the difference between function overloading and function overriding?

Assignment No. 4

Title	Write a C++ program to demonstrate runtime polymorphism by creating a base class Shape with a virtual function to calculate area. Implement derived classes like Rectangle and Circle that override the base class function.
Aim/Problem Statement	Design and implement a C++ program to demonstrate **runtime polymorphism** by creating a base class named Employee with a **virtual function** to calculate and display salary details. Derive classes such as PermanentEmployee and ContractEmployee from the base class and override the virtual function to implement their respective salary calculations. Use a **base class pointer** to invoke the overridden functions at runtime, thereby demonstrating **runtime polymorphism**, **function overriding**, and **code reusability** in an employee management system.
CO Mapped	**CO 2**
Pre-requisite	• Basics of C++ programming • Structure of a C++ program • Classes and Objects • Data Members and Member Functions

	• Access Specifiers (`private`, `protected`, and `public`) • Constructors and Destructors • Function Overloading and Function Overriding • Inheritance (Single Inheritance) • Virtual Functions • Runtime Polymorphism • Base Class and Derived Class concepts • Object creation and member function invocation • Input and Output using `cin` and `cout` • Fundamentals of Object-Oriented Programming (OOP), including: 1. Encapsulation 2. Inheritance 3. Polymorphism 4. Code Reusability
Learning Objective	1. Understand the concept of **runtime polymorphism** in C++. 2. Implement **virtual functions** in a base class. 3. Override virtual functions in derived classes to provide specialized implementations. 4. Use **base class pointers** to invoke overridden functions dynamically at runtime. 5. Differentiate between **compile-time polymorphism** and **runtime polymorphism**. 6. Demonstrate the relationship between **inheritance** and **polymorphism** in Object-Oriented Programming. 7. Apply OOP concepts such as **inheritance**, **function overriding**, **virtual functions**, and **runtime polymorphism** to develop a real-world employee management application. 8. Appreciate the benefits of runtime polymorphism in achieving **flexibility**, **extensibility**, and **maintainability** in software design.

Theory:

Introduction

Polymorphism is one of the fundamental principles of **Object-Oriented Programming (OOP)**. The word *polymorphism* is derived from the Greek words **"poly"**, meaning *many*, and **"morph"**, meaning *forms*. In C++, polymorphism allows a single interface to represent different implementations, enabling the same function call to perform different tasks depending on the type of object.

Polymorphism improves **code reusability**, **flexibility**, **maintainability**, and **extensibility**, making software easier to develop and manage.

This experiment demonstrates **runtime polymorphism** using **virtual functions** by creating a base class Employee and derived classes PermanentEmployee and ContractEmployee. Each derived class provides its own implementation of the salary calculation function.

Runtime Polymorphism

Runtime polymorphism is the process in which the function to be executed is determined **during program execution** rather than during compilation.

It is achieved using:

- Inheritance
- Virtual Functions
- Function Overriding
- Base Class Pointer or Reference

The decision about which function to call is made at **runtime**, depending on the object being referenced.

Virtual Function

A **virtual function** is a member function declared in the base class using the keyword **virtual**. It allows derived classes to provide their own implementation of the function.

When a virtual function is called using a **base class pointer or reference**, C++ executes the function belonging to the actual object instead of the pointer type.

Syntax

```
class Employee
{
public:
   virtual void calculateSalary()
   {
      cout << "Salary";
   }
};
```

Function Overriding

Function overriding occurs when a derived class defines a member function with the **same name**, **same return type**, and **same parameter list** as a virtual function in the base class.

The overridden function replaces the base class implementation for objects of the derived class.

Example:

```
class PermanentEmployee : public Employee
{
public:
   void calculateSalary()
   {
      // Salary calculation for permanent employee
   }
};
```

Base Class Pointer

A base class pointer can point to objects of both the base class and its derived classes.

Example:

Employee *ptr;

PermanentEmployee p;

ptr = &p;

ptr->calculateSalary();

Because calculateSalary() is declared as a virtual function, the overridden function in PermanentEmployee is executed.

Dynamic Binding (Late Binding)

Binding is the process of associating a function call with the function definition.

There are two types of binding:

1. Early Binding (Static Binding)

- Performed during compilation.
- Used for normal member functions.
- Faster execution.

2. Late Binding (Dynamic Binding)

- Performed during runtime.
- Achieved using virtual functions.
- Supports runtime polymorphism.

In this experiment, **late binding** is used to invoke the appropriate salary calculation function based on the object type.

Role of Inheritance

Inheritance is essential for runtime polymorphism.

- The base class defines a common interface.
- Derived classes inherit the common members.
- Derived classes override virtual functions according to their requirements.

This allows different employee types to calculate salary differently while sharing common employee information.

Algorithm:

1. **Start.**

2. Define a **base class** Employee with the following data members:
3. Employee ID
4. Employee Name
5. Basic Salary
6. Declare a **virtual member function** calculateSalary() in the base class.
7. Define a member function accept() to input employee details.
8. Define a member function display() to display employee details.
9. Create a **derived class** PermanentEmployee that inherits from Employee.
10. Override the calculateSalary() function in PermanentEmployee to calculate the salary by adding allowances such as HRA and DA to the basic salary.
11. Create another **derived class** ContractEmployee that inherits from Employee.
12. Override the calculateSalary() function in ContractEmployee to calculate the salary according to the contract employee's pay structure.
13. Create objects of both derived classes.
14. Declare a **base class pointer** of type Employee.
15. Assign the address of the PermanentEmployee object to the base class pointer.
16. Call the accept(), display(), and calculateSalary() functions using the base class pointer.
17. Reassign the base class pointer to the ContractEmployee object.
18. Again call the accept(), display(), and calculateSalary() functions through the base class pointer.
19. Observe that the appropriate overridden calculateSalary() function is invoked at runtime based on the object type, demonstrating **runtime polymorphism**.
20. **Stop.**

Flow chart (Students Should Draw Flowchart Here)

Conclusion:

Runtime polymorphism is a powerful feature of C++ that enables programs to select the appropriate function implementation during execution. By combining **inheritance**, **virtual functions**, **function overriding**, and **base class pointers**, developers can create flexible, reusable, and maintainable software. In this experiment, different employee types calculate salaries through their own implementations while sharing a common interface, demonstrating the practical use of runtime polymorphism in real-world applications.

Questions:

1. What is the difference between compile-time and runtime polymorphism?

2. What is a virtual function?
3. What is function overriding?
4. Why is a base class pointer used in runtime polymorphism?

Assignment No. 5

Title	Design a C++ program to perform arithmetic division of two numbers. Handle exceptions such as division by zero and invalid input using exception handling mechanisms.
Aim/Problem Statement	Design and implement a C++ program to perform the **division of two numbers** entered by the user. The program should use

	exception handling mechanisms to detect and handle runtime errors such as **division by zero** and **invalid input**. Display appropriate error messages when an exception occurs and ensures that the program terminates gracefully without crashing.
CO Mapped	CO 3
Pre -requisite	• Basics of C++ programming • Structure of a C++ program • Variables, Data Types, and Operators • Input and Output using cin and cout • Conditional Statements (if, if-else) • Functions in C++ • Runtime Errors and Logical Errors • Basics of **Exception Handling** • try, throw, and catch blocks • Standard Exception Classes (basic understanding) • Fundamentals of Object-Oriented Programming (OOP) concepts in C++ (basic knowledge)
Learning Objective	1. Understand the concept and importance of **exception handling** in C++. 2. Identify common **runtime errors**, such as division by zero and invalid input. 3. Use **try**, **throw**, and **catch** blocks to handle exceptions effectively. 4. Implement user-defined and built-in exception handling mechanisms. 5. Develop robust programs that can detect and respond to exceptional conditions without abrupt termination. 6. Display meaningful error messages and ensure graceful program execution when exceptions occur. 7. Differentiate between **syntax errors**, **logical errors**, and **runtime errors**. 8. Apply exception handling techniques to improve the **reliability**, **robustness**, and **maintainability** of C++ applications.

Theory:

During program execution, unexpected situations such as invalid input, division by zero, file access errors, or memory allocation failures may occur. These situations are known as **exceptions**. If exceptions are not handled properly, the program may terminate abnormally, leading to unreliable software.

Exception Handling is a mechanism provided by C++ to detect, throw, and handle runtime errors. It separates normal program logic from error-handling logic, making programs more robust, maintainable, and user-friendly.

In this experiment, exception handling is used to safely perform arithmetic division by handling two common errors:

- Division by zero
- Invalid (non-numeric) input

Types of Errors in C++

1. Syntax Errors

These occur due to incorrect grammar or syntax in the program and are detected during compilation.

Example:

```
int a = 10
```

(Missing semicolon)

2. Logical Errors

These errors occur when the program compiles successfully but produces incorrect results due to flawed logic.

Example:
Using `+` instead of `*` in a multiplication calculation.

3. Runtime Errors

These occur while the program is executing.

Examples include:

- Division by zero
- Invalid user input
- File not found
- Memory allocation failure
- Array index out of bounds (in some contexts)

Exception handling is mainly used to manage **runtime errors**.

Exception Handling in C++

Exception handling is implemented using three keywords:

1. `try`

The code that may generate an exception is placed inside a try block.

Syntax:

```
try
{
   // Code that may throw an exception
}
```

2. throw

The throw statement is used to generate an exception when an abnormal condition occurs.

Syntax:

throw exception;

Example:

```
if(denominator == 0)
{
   throw "Division by Zero";
}
```

3. catch

A catch block receives and handles the exception thrown from the try block.

Syntax:

```
catch(type variable)
{
   // Exception handling code
}
```

Working of Exception Handling

1. The statements inside the try block execute normally.
2. If an exception occurs, the throw statement transfers control to the matching catch block.
3. The catch block handles the exception.
4. After handling the exception, the program continues execution after the catch block.

Division by Zero Exception

Division by zero is one of the most common runtime errors.

Example:

```
if(divisor == 0)
{
   throw "Division by Zero is not allowed.";
}
```

The exception is caught and an appropriate error message is displayed.

Invalid Input Handling

If the user enters a non-numeric value instead of a number, the input stream enters a failure state.

This can be checked using:

```
if(cin.fail())
{
   throw "Invalid Input";
}
```

The exception informs the user to enter valid numeric data.

Advantages of Exception Handling

- Prevents abnormal program termination.
- Separates error-handling code from normal program logic.
- Improves program readability.
- Makes applications more reliable.
- Simplifies debugging and maintenance.
- Enhances software robustness.

Algorithm:

1. **Start.**
2. Declare two variables for the dividend and divisor.
3. Accept both numbers from the user.
4. Check whether the input is valid.
5. If invalid input is detected, throw an exception.
6. Enter the `try` block.
7. Check whether the divisor is zero.
8. If the divisor is zero, throw an exception.
9. Otherwise, perform the division.
10. Display the result.
11. Catch and handle exceptions for:
12. Division by zero
13. Invalid input
14. Display an appropriate error message.
15. **End the program.**

Flowchart:(Students Should Draw flowchart here)

Conclusion:

This experiment demonstrates the implementation of **exception handling** in C++. By using the try, throw, and catch mechanism, the program safely handles runtime errors without terminating unexpectedly. Exception handling improves the reliability, robustness, and maintainability of software by enabling graceful error recovery and meaningful communication with the user.

Questions:

1. What is the function of the `throw` statement?
2. What is the role of the `catch` block?
3. What happens if an exception is not caught?
4. How can `cin.fail()` be used to detect invalid input?

Assignment No. 6

Title	Create a C++ program to store and retrieve records of employees using file handling. Implement a class Employee and perform write and read operations on files using C++ file streams.
Aim/Problem Statement	Design and implement a C++ program to create a class named Employee with data members such as **employee ID**, **employee name**, **designation**, and **salary**. Use **file handling** to store employee records in a file and retrieve them when required. Implement member functions to perform **write** and **read** operations using C++ file streams (ofstream and ifstream). Demonstrate the use of file handling for persistent storage and retrieval of employee information.
CO Mapped	**CO 3**
Pre -requisite	• Basics of C++ programming • Structure of a C++ program • Classes and Objects • Data Members and Member Functions • Constructors and Objects • Input and Output using `cin`, `cout`, and `getline()` • String data type (`string`) • File Handling concepts • C++ File Stream Classes (`ifstream`, `ofstream`, and `fstream`) • Opening, reading, writing, and closing files • Fundamentals of Object-Oriented Programming (OOP) • Basic understanding of text files and persistent data storage
Learning Objective	1. Understand the concept and importance of **file handling** in C++. 2. Create a class to represent employee records using appropriate data members and member functions. 3. Use **C++ file stream classes** (`ifstream`, `ofstream`, and `fstream`) to perform file operations. 4. Write employee records to a file and retrieve them using read operations. 5. Implement **persistent storage** for employee information using files. 6. Apply Object-Oriented Programming (OOP) concepts along with file handling to develop a simple employee record management system. 7. Handle file opening, closing, and basic file access operations correctly.

	8. Appreciate the role of file handling in storing and managing data for real-world applications such as payroll, employee management, and database systems.

Theory:

INTRODUCTION

File handling is an important feature of C++ that allows programs to store data permanently in files. Unlike variables, whose contents are lost when the program terminates, data stored in files remains available until it is modified or deleted.

File handling is widely used in applications such as employee management systems, banking systems, hospital management systems, inventory management systems, and student information systems, where records need to be stored and retrieved efficiently.

In C++, file operations are performed using the **fstream** library, which provides specialized classes for reading from and writing to files.

Need for File Handling

Without file handling:

- Data is stored temporarily in memory.
- Information is lost when the program ends.
- Large datasets cannot be managed efficiently.

With file handling:

- Data is stored permanently.
- Records can be retrieved whenever required.
- Applications become more reliable and practical.

File Stream Classes

C++ provides three classes for file handling through the <fstream> header file.

1. ifstream (Input File Stream)

Used to read data from a file.

ifstream fin("Employee.txt");

2. ofstream (Output File Stream)

Used to write data into a file.

ofstream fout("Employee.txt");

3. fstream

Used for both reading and writing.

fstream file("Employee.txt");

Opening a File

A file can be opened using the constructor or the open() function.

```
ofstream fout;
fout.open("Employee.txt");
```

Writing to a File

Data is written using the insertion (<<) operator.

```
fout << empId << endl;
fout << empName << endl;
```

Reading from a File

Data is read using the extraction (>>) operator or getline().

```
fin >> empId;
getline(fin, empName);
```

Closing a File

After completing file operations, the file should be closed.

```
fout.close();
fin.close();
```

Closing the file ensures that all buffered data is written to disk and releases system resources.

Advantages of File Handling

- Permanent data storage
- Efficient record management
- Data can be reused later
- Easy retrieval of information
- Reduces manual record maintenance
- Supports large applications

Applications

File handling is widely used in:

- Employee Management Systems
- Payroll Systems
- Banking Applications
- Hospital Management Systems

- Library Management Systems
- Inventory Management Systems
- School and College Management Systems
- Online Shopping Systems
- Database Applications

Algorithm:

1. Start.
2. Define a class Employee.
3. Declare data members:

- Employee ID
- Employee Name
- Designation
- Salary

4. Create a member function accept() to input employee details.
5. Open a file using ofstream.
6. Write employee details to the file.
7. Close the file.
8. Open the same file using ifstream.
9. Read employee details from the file.
10. Display the retrieved employee records.
11. Close the file.
12. Stop.

Flowchart :(Students should draw flowchart here)

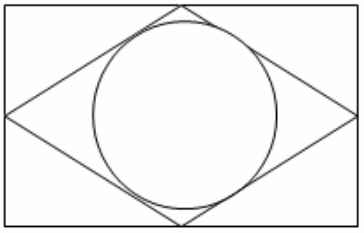
Conclusion:

This experiment demonstrates the implementation of **file handling** in C++ using the Employee class. It illustrates how employee records can be stored permanently in a file and retrieved whenever required using C++ file streams. The experiment highlights the importance of persistent data storage and the role of file handling in developing practical applications such as employee management and payroll systems.

Questions:

1. What is the purpose of the <fstream> header file?
2. How do you open and close a file in C++?
3. What is the difference between text files and binary files?
4. What happens if a file cannot be opened??

Assignment No. 7

Title	Write a C++ Program to draw the following pattern using The DDA line and Bresenham's Circle drawing algorithm. 
Aim/Problem Statement	Design and implement a C++ program to draw the given graphical pattern by combining the **Digital Differential Analyzer (DDA) Line Drawing Algorithm** and the **Bresenham's Circle Drawing Algorithm**. The program should use the DDA algorithm to draw straight lines and Bresenham's algorithm to draw circles accurately and efficiently. Display the required pattern on the graphics screen by integrating both algorithms and demonstrate the fundamental principles of computer graphics rasterization techniques.
CO Mapped	CO4
Pre -requisite	• Basics of C++ programming

	• Arrays and Mathematical Functions • Fundamentals of Computer Graphics • Coordinate System and Pixel Representation • Line Drawing Concepts • Circle Drawing Concepts • Digital Differential Analyzer (DDA) Line Drawing Algorithm • Bresenham's Circle Drawing Algorithm • Basic knowledge of graphics libraries (such as `graphics.h`/WinBGIm or an equivalent graphics library) • Basic programming skills of C++ • 64-bit Open source Linux • Open Source C++ Programming tool like G++/GCC
Learning Objective	1. Understand the fundamentals of **computer graphics** and raster graphics. 2. Implement the **Digital Differential Analyzer (DDA) Line Drawing Algorithm** to draw straight lines. 3. Implement the **Bresenham's Circle Drawing Algorithm** to draw circles efficiently. 4. Apply line and circle drawing algorithms to construct a given graphical pattern. 5. Understand the concept of **pixel plotting** and coordinate systems in computer graphics. 6. Compare the characteristics and efficiency of the DDA line drawing algorithm and Bresenham's circle drawing algorithm. 7. Develop graphical applications using C++ and a graphics library (such as `graphics.h`/WinBGIm or an equivalent graphics library). 8. Enhance problem-solving skills by integrating multiple computer graphics algorithms to create complex graphical objects and patterns.

Theory:

DDA Line Drawing Algorithm:

Line is a basic element in graphics. To draw a line, you need two end points between which you can draw a line. Digital Differential Analyzer (DDA) line drawing algorithm is the simplest line drawing algorithm in computer graphics. It works on an incremental method. It plots the points from starting point of line to end point of line by incrementing in X and Y direction in each iteration.
DDA line drawing algorithm works as follows:

Step 1: Get coordinates of both the end points (X_1, Y_1) and (X_2, Y_2) from the user.

Step 2: Calculate the difference between two end points in X and Y direction.

$dx = X_2 - X_1;$
$dy = Y_2 - Y_1;$

Step 3: Based on the calculated difference in step-2, you need to identify the number of steps to put pixel. If dx > dy, then you need more steps in x coordinate; otherwise in y coordinate.

if (absolute(dx) > absolute(dy))

```
Steps = absolute(dx);
else
Steps = absolute(dy);
```

Step 4: Calculate the increment in x coordinate and y coordinate.

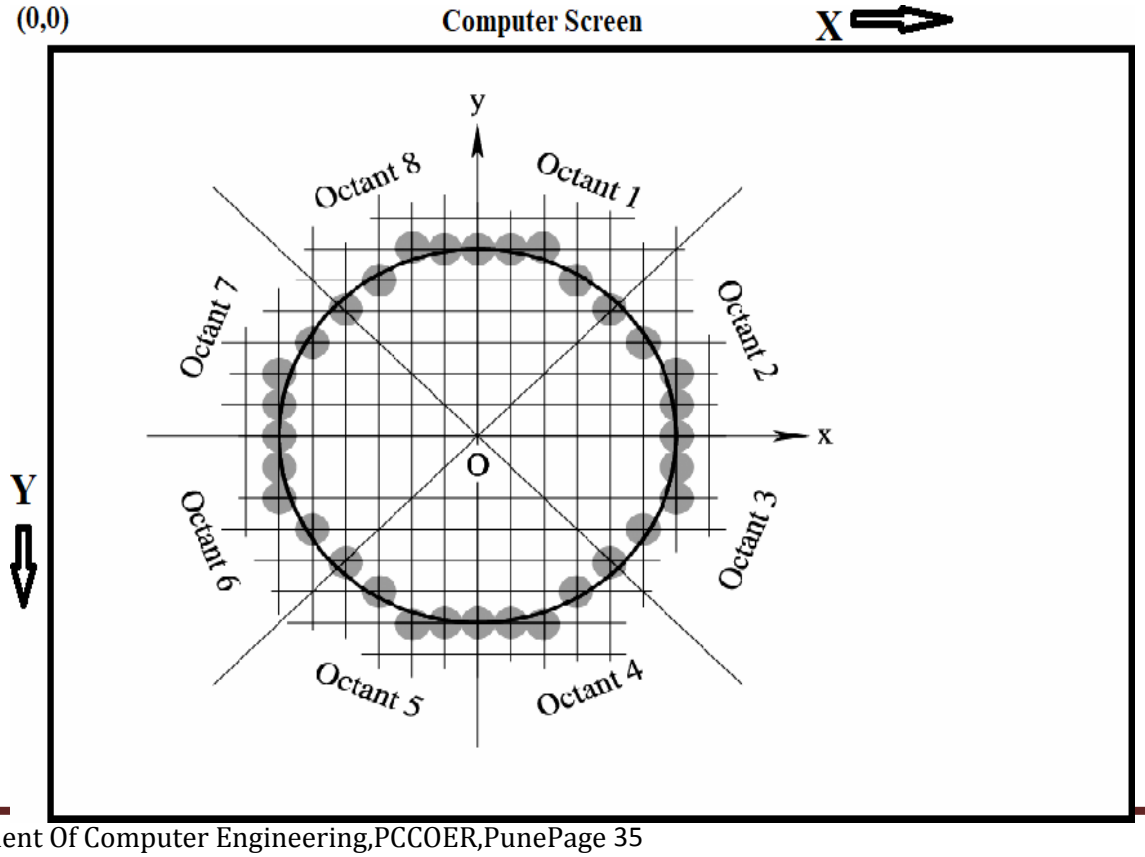
```
Xincrement = dx / (float) steps;
Yincrement = dy / (float) steps;
```

Step 5: Plot the pixels by successfully incrementing x and y coordinates accordingly and complete the drawing of the line.

```
for(int i=0; i < Steps; i++)
{
        X1 = X1 + Xincrement;
        Y1 = Y1 + Yincrement;
        putpixel(Round(X1), Round(Y1), ColorName);
}
```

2) Bresenham's Circle Drawing Algorithm:

A circle is an eight-way symmetric figure. The shape of the circle is the same in all quadrants. In each quadrant, there are two octants. If the calculation of the point of one octant is done, then the other seven points can be calculated easily by using the concept of eight-way symmetry.
Bresenham's Circle Drawing Algorithm is a circle drawing algorithm that selects the nearest pixel position to complete the arc. The unique part of this algorithm is that it uses only integer arithmetic which makes it significantly faster than other algorithms using floating point arithmetic.



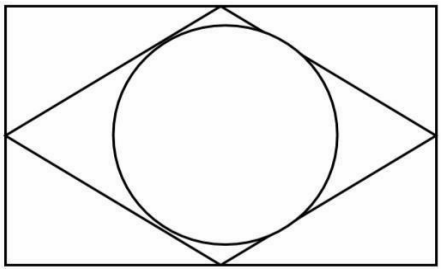
Step 1: Read the x and y coordinates of center: (centx, centy)
Step 2: Read the radius of circle: (r)
Step 3: Initialize, x = 0; y = r;
Step 4: Initialize decision parameter: p = 3 – (2 * r)
Step 5:

```
do {
putpixel(centx+x, centy-y, ColorName);
if (p<0)
p = p+(4*x)+6;
else
{
p=p+[4*(x-y)]+10;
y=y-1;
}
x=x+1;
} while(x<y)
```

Here in step 5, putpixel() function is used which will print Octant-1 of the circle with radius r after the completion of all the iterations of the do-while loop. Because of the eight-way symmetry property of circle, we can draw other octants of the circle using following putpixel() function:

Octant 1: putpixel(centx+x, centy-y, ColorName);
Octant 2: putpixel(centx+y, centy-x, ColorName);
Octant 3: putpixel(centx+y, centy+x, ColorName);
Octant 4: putpixel(centx+x, centy+y, ColorName);
Octant 5: putpixel(centx-x, centy+y, ColorName);
Octant 6: putpixel(centx-y, centy+x, ColorName);
Octant 7: putpixel(centx-y, centy-x, ColorName);
Octant 8: putpixel(centx-x, centy-y, ColorName);

Drawing Pattern using lines and circles:



To draw this pattern, we require two rectangles and one circle. We can use suitable geometry to get coordinates to draw rectangles and circle. Then we can call DDA line drawing and Bresenham's circle drawing algorithms by passing appropriate parameters to get the required pattern.

Conclusion:
This experiment demonstrates the implementation of two fundamental raster graphics algorithms in

C++. The DDA Line Drawing Algorithm is used to draw straight lines by calculating intermediate pixel positions, while Bresenham's Circle Drawing Algorithm efficiently generates circles using integer arithmetic and eight-way symmetry. By combining these algorithms, complex graphical patterns can be created accurately and efficiently, providing a strong foundation for understanding computer graphics and graphics programming.

Questions:

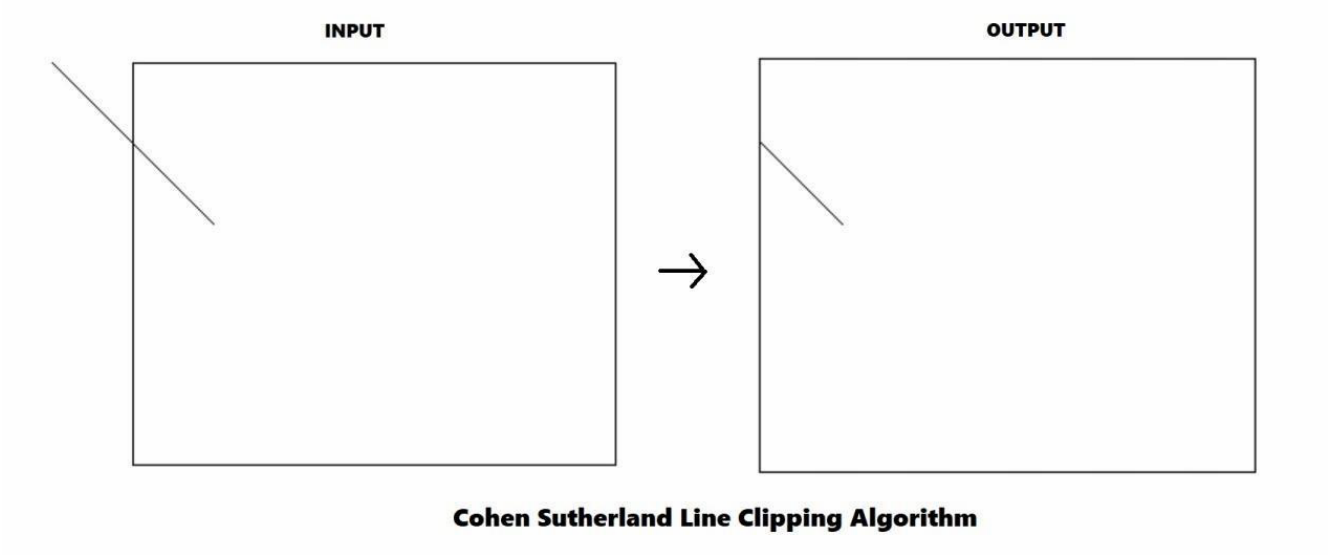
1. Explain eight-way symmetric figure for circle drawing.
2. Explain the DDA line drawing algorithm.
3. What is step in DDA line drawing algorithm?
4. List some real-world applications of line and circle drawing algorithms.

Assignment No. 8

Title	Write a C++ program to implement the Cohen–Sutherland line clipping algorithm for clipping a line against a rectangular clipping window.
Aim/Problem Statement	Design and implement a C++ program to perform **line clipping** using the **Cohen–Sutherland Line Clipping Algorithm**. The program should accept the endpoints of a line segment and the coordinates of a rectangular clipping window. Determine whether the line is **completely visible, completely invisible, or partially visible** with respect to the clipping window. If the line is partially visible, compute the clipped line segment and display the original as well as the clipped line using a graphics library. This program should demonstrate the principles of clipping and efficient rendering in computer graphics.
CO Mapped	**CO 5**
Pre -requisite	1. Basic programming skills of C++ 2. 64-bit Open source Linux 3. Open Source C++ Programming tool like G++/GCC
Learning Objective	1. Understand the concept of **clipping** in computer graphics. 2. Explain the working principle of the **Cohen–Sutherland Line Clipping Algorithm**. 3. Generate and interpret **region codes (outcodes)** for line endpoints. 4. Classify a line as **completely visible**, **completely invisible**, or **partially visible**. 5. Compute the intersection points of a line with the clipping window boundaries. 6. Implement the Cohen–Sutherland algorithm using C++ and a graphics library. 7. Visualize the original and clipped line segments. 8. Appreciate the importance of clipping in improving graphics rendering efficiency.

Theory:

Cohen Sutherland Algorithm is a **line clipping algorithm** that cuts lines to portions which are within a rectangular area. It eliminates the lines from a given set of lines and rectangle area of interest (view port) which belongs outside the area of interest and clips those lines which are partially inside the area of interest.

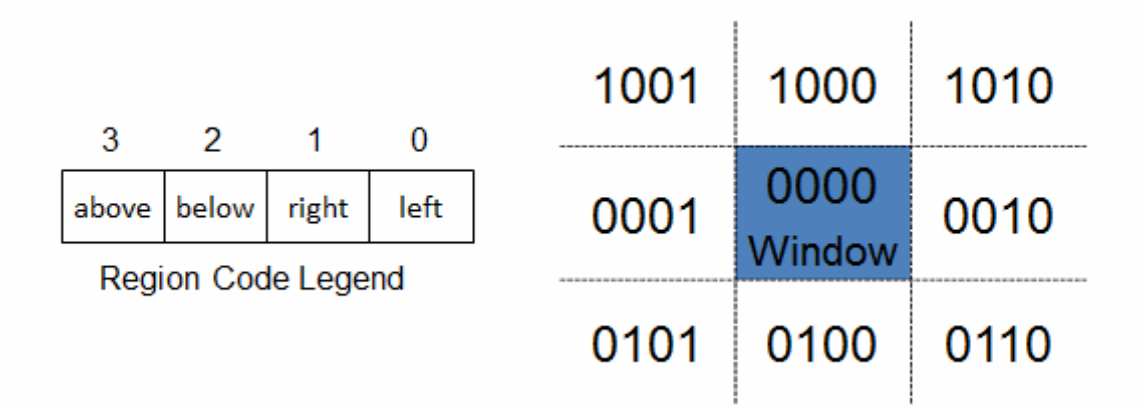


Cohen Sutherland Line Clipping Algorithm

Algorithm

The algorithm divides **a two-dimensional space** into **9 regions** (eight outside regions and one inside region) and then efficiently determines the lines and portions of lines that are visible in the central region of interest (the viewport).

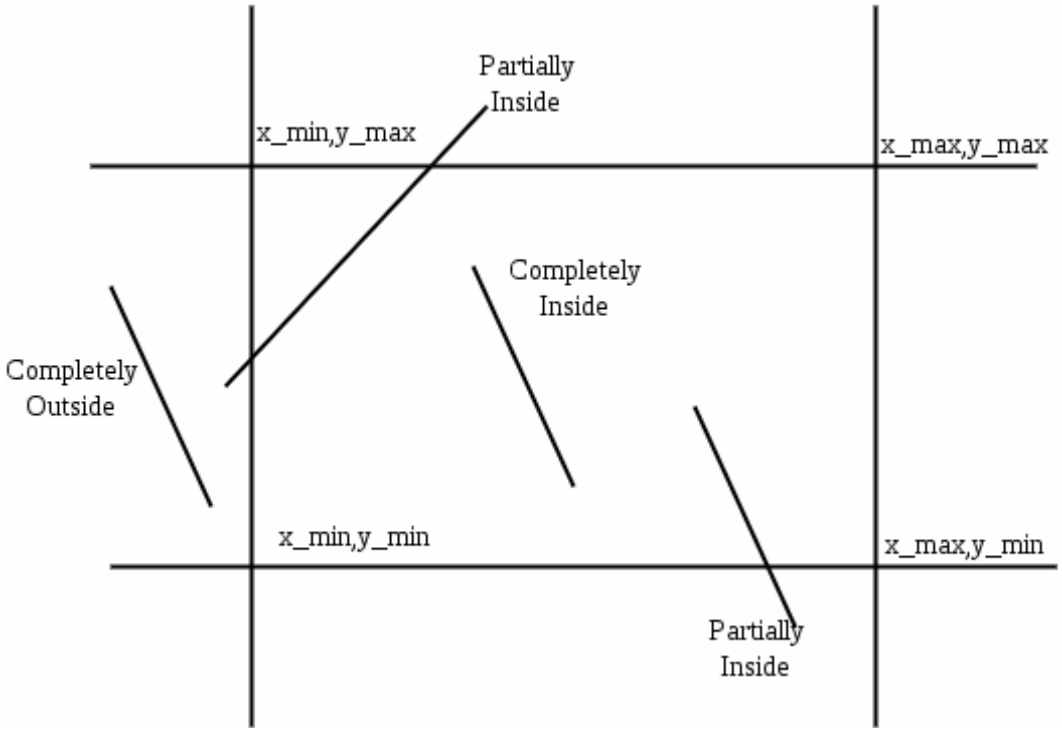
Following image illustrates the 9 regions:



As you seen each region is denoted by a 4 bit code like 0101 for the bottom right region

Four Bit code is calculated by comparing extreme end point of given line (x,y) by four co-ordinates x_min, x_max, y_max, y_min which are the coordinates of the area of interest (0000)

1. Set First Bit if 1 Points lies to left of window (**x < x_min**)
2. Set Second Bit if 1 Points lies to right of window (**x > x_max**)
3. Set Third Bit if 1 Points lies to left of window (**y < y_min**)
4. Set Forth Bit if 1 Points lies to right of window (y > **y_max**)



The more efficient Cohen-Sutherland Algorithm performs initial tests on a line to determine whether intersection calculations can be avoided.

Advantages of Cohen–Sutherland Algorithm

- Fast and efficient.
- Easy implementation.
- Uses simple bitwise operations.
- Reduces unnecessary calculations.
- Suitable for rectangular clipping windows.

Limitations

- Applicable only to rectangular clipping windows.
- Not suitable for arbitrary polygon clipping.
- Requires repeated calculations for long line segments crossing multiple boundaries.

Applications

The Cohen–Sutherland algorithm is used in:

- Computer-Aided Design (CAD)
- Computer Games
- Geographic Information Systems (GIS)
- Windowing Systems
- Graphics Editors
- Scientific Visualization
- Image Processing
- Simulation Software

Pseudocode

- **Step 1** : Assign a region code for two endpoints of given line
- **Step 2:** If both endpoints have a region code 0000 then given line is completely inside and we will keep this line.
- **Step 3:** If step 2 fails, perform the logical AND operation for both region codes.

  Step 3.1: If the result is not 0000, then given line is completely outside.

  Step 3.2: Else line is partially inside.

  Step 3.2.a: Choose an endpoint of the line that is outside the given rectangle.

  Step 3.2.b : Find the intersection point of the rectangular boundary(based on region code)

  Step 3.2.c: Replace endpoint with the intersection point and upgrade the region Code.

  Step 3.2.d: Repeat step 2 until we find a clipped line either trivially accepted or Rejected.
- **Step 4:** Repeat step 1 for all lines.

Conclusion:

This experiment demonstrates the implementation of the **Cohen–Sutherland Line Clipping Algorithm**, an efficient algorithm for clipping line segments against a rectangular window. By using **4-bit region codes** and **bitwise operations**, the algorithm minimizes unnecessary calculations and quickly determines the visibility of a line. It forms the basis for many modern computer graphics systems where efficient rendering of visible objects is essential.

Questions:

1. What is the limitation of Cohen Sutherland Line Clipping algorithm?
2. What are the advantages of Cohen Sutherland Line Clipping?
3. What is clipping in computer graphics?
4. Why is clipping required?

Assignment No. 9

Title	Write a **C++ program** to implement **2-D transformations** such as translation, scaling, rotation, reflection, and shearing on a given object.
Aim/Problem Statement	Design and implement a C++ program to perform **2-D geometric transformations** on a given object. The program should allow the user to apply transformations such as **translation, scaling, rotation, reflection, and shearing** on the object using appropriate transformation matrices. Display both the **original object** and the **transformed object** using a graphics library to visualize the effect of each transformation. This program should demonstrate the mathematical principles of **2-D transformations** used in computer graphics.
CO Mapped	**CO 5**
Pre -requisite	4. Basic programming skills of C++ 5. 64-bit Open source Linux 6. Open Source C++ Programming tool like G++/GCC
Learning Objective	1. Understand the fundamentals of **2-D geometric transformations**. 2. Implement translation, scaling, rotation, reflection, and shearing transformations. 3. Apply transformation matrices to graphical objects. 4. Display the original and transformed objects using graphics functions. 5. Understand homogeneous coordinates and matrix representation of transformations. 6. Compare different transformation techniques. 7. Develop graphics applications using C++. 8. Apply geometric transformations in CAD, animation, gaming, image processing, and GUI applications.

Theory:

Geometric transformations are fundamental operations in **Computer Graphics** that modify the position, size, orientation, or shape of graphical objects. These transformations are widely used in computer-aided design (CAD), animation, image processing, game development, robotics, and graphical user interfaces.A two-dimensional (2-D) object is represented by a set of coordinate points in

the Cartesian plane. By applying mathematical transformation matrices to these coordinates, the object can be moved, resized, rotated, reflected, or skewed while preserving its geometric relationships.

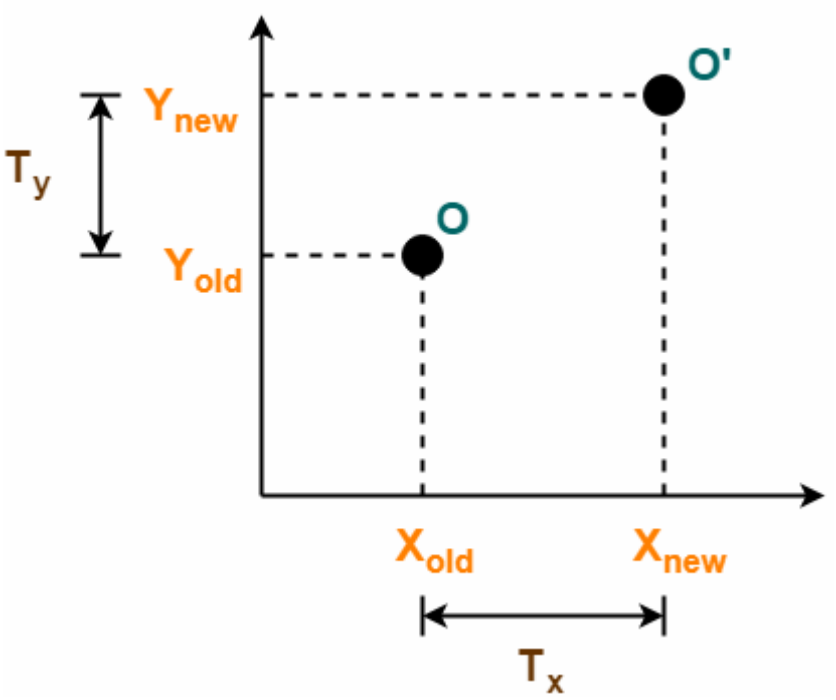
The five basic 2-D transformations are:

- Translation
- Scaling
- Rotation
- Reflection
- Shearing

1) Translation:

A translation moves an object to a different position on the screen. You can translate a point in 2D by adding translation coordinate or translation vector (T_x, T_y) to the original coordinates. Consider

- Initial coordinates of the object O = (X_{old}, Y_{old})
- New coordinates of the object O after translation = (X_{new}, Y_{new})
- Translation vector or Shift vector = (T_x, T_y)



This translation is achieved by adding the translation coordinates to the old coordinates of the object as-

1. Xnew = Xold + Tx (This denotes translation towards X axis)
2. Ynew = Yold + Ty (This denotes translation towards Y axis)

In Matrix form, the above translation equations may be represented as-

$$\begin{bmatrix} X_{new} \\ Y_{new} \end{bmatrix} = \begin{bmatrix} X_{old} \\ Y_{old} \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix}$$

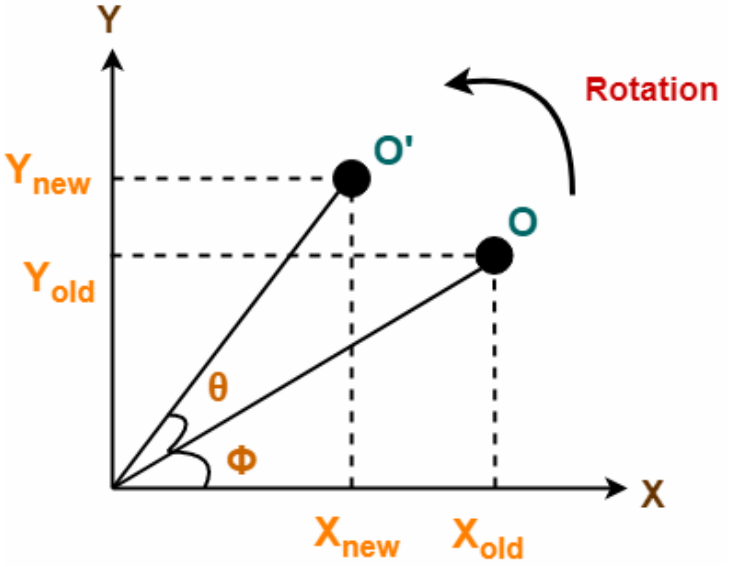
Translation Matrix

2)

2) Rotation:

In rotation, we rotate the object at particular angle θ (theta) from its original position. Consider
In Matrix form, the above translation equations may be represented as-

- Initial coordinates of the object O = (Xold, Yold)
- Initial angle of the object O with respect to origin = Φ
- Rotation angle = θ
- New coordinates of the object O after rotation = (Xnew, Ynew)



This anti-clockwise rotation is achieved by using the following rotation equations-

$$X_{new} = X_{old} \times \cos\theta - Y_{old} \times \sin\theta$$
$$Y_{new} = X_{old} \times \sin\theta + Y_{old} \times \cos\theta$$

In Matrix form, the above rotation equations may be represented as-

$$\begin{bmatrix} X_{new} \\ Y_{new} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \end{bmatrix}$$

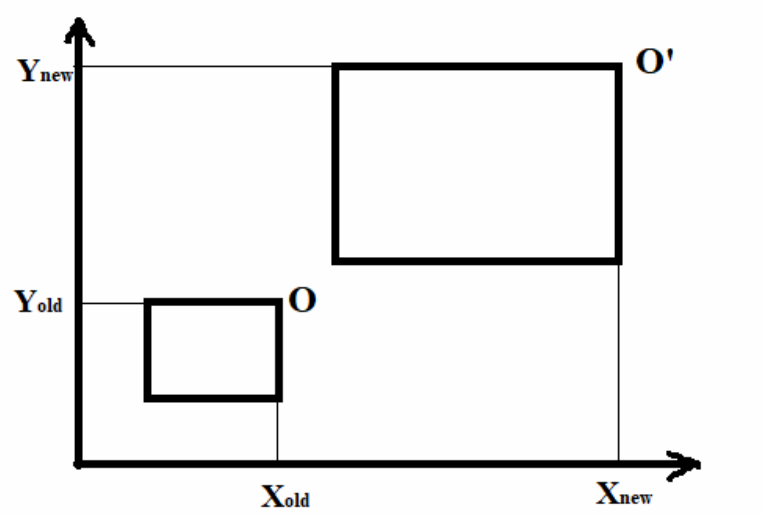
Rotation Matrix

3) Scaling:

Scaling transformation is used to change the size of an object. In the scaling process, you either expand or compress the dimensions of the object. Scaling can be achieved by multiplying the original

coordinates of the object with the scaling factor (Sx, Sy). If scaling factor > 1, then the object size is increased. If scaling factor < 1, then the object size is reduced. Consider

- Initial coordinates of the object O = (Xold, Yold)
- Scaling factor for X-axis = Sx
- Scaling factor for Y-axis = Sy
- New coordinates of the object O after scaling = (Xnew, Ynew)



This scaling is achieved by using the following scaling equations- Xnew = Xold x Sx
Ynew = Yold x Sy
In Matrix form, the above scaling equations may be represented as-

$$\begin{bmatrix} X_{new} \\ Y_{new} \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \end{bmatrix}$$

Scaling Matrix

Homogeneous Coordinates:

Matrix multiplication is easier to implement in hardware and software as compared to matrix addition. Hence we want to replace matrix addition by multiplication while performing transformation operations. So the solution is **homogeneous coordinates** , which allows us to express all transformations (including translation) as matrix multiplications.

To obtain homogeneous coordinates we have to represent transformation matrices in 3x3 matrices instead of 2x2. For this we add dummy coordinate. Each 2 dimensional position (x,y) can be represented by homogeneous coordinate as (x,y,1).

Translation Matrix (Homogeneous Coordinates representation)

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{bmatrix} x \begin{bmatrix} X_{old} \\ Y_{old} \\ 1 \end{bmatrix}$$

Rotation Matrix (Homogeneous Coordinates representation)

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} x \begin{bmatrix} X_{old} \\ Y_{old} \\ 1 \end{bmatrix}$$

Scaling Matrix (Homogeneous Coordinates representation)

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} x \begin{bmatrix} X_{old} \\ Y_{old} \\ 1 \end{bmatrix}$$

Applying transformations on equilateral triangle:

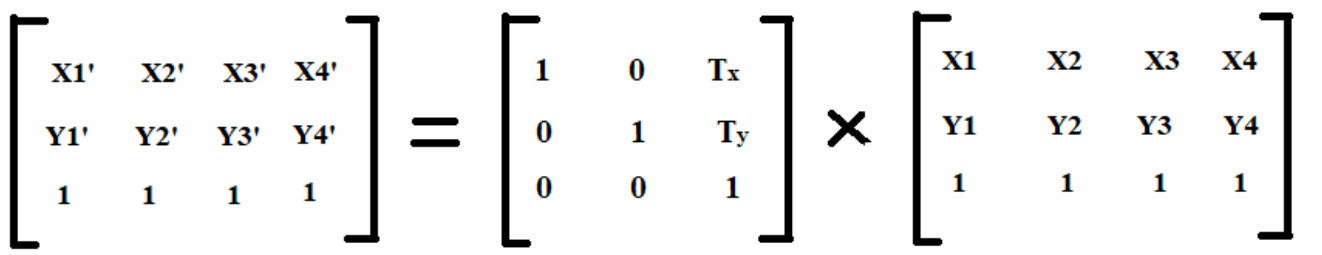
Consider that coordinates of vertices of equilateral triangle are (X1,Y1), (X2,Y2) and (X3,Y3). After applying basic transformations, we will get corresponding coordinates as (X1',Y1'), (X2',Y2') and (X3',Y3') respectively. Following multiplication will give the translation on this equilateral **triangle:**

$$\begin{bmatrix} X1' & X2' & X3' \\ Y1' & Y2' & Y3' \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & Tx \\ 0 & 1 & Ty \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X1 & X2 & X3 \\ Y1 & Y2 & Y3 \\ 1 & 1 & 1 \end{bmatrix}$$

Similarly we can apply rotation and scaling on equilateral triangle.

Applying transformations on rhombus:

Consider that coordinates of vertices of rhombus are (X1,Y1), (X2,Y2), (X3,Y3) and (X4,Y4) applying basic transformations, we will get corresponding coordinates as (X1',Y1'), (X2',Y2'), (X3',Y3') and (X4',Y4') respectively. Following multiplication will give the translation on this rhombus:



Similarly we can apply rotation and scaling on rhombus.

Conclusion:

This experiment demonstrates the implementation of the fundamental **2-D geometric transformations** used in computer graphics. By applying transformation matrices, graphical objects can be translated, scaled, rotated, reflected, and sheared efficiently. These operations form the basis of many graphics applications, including animation, CAD, gaming, image processing, and user interface design, highlighting the importance of matrix-based transformations in graphical programming.

Questions:

1. What are geometric transformations in computer graphics?
2. What is the purpose of homogeneous coordinates?
3. What is a transformation matrix?
4. Differentiate between translation and scaling.

Assignment No. 10

Title	Write a **C++ program** to implement **rotation of a 2-D object about an arbitrary point**.
Aim/Problem Statement	Design and implement a C++ program to perform the **rotation of a 2-D object about an arbitrary point**. The program should accept the coordinates of the object, the coordinates of the arbitrary point of rotation, and the angle of rotation from the user. Apply the appropriate transformation matrices to rotate the object about the specified point and display both the **original** and the **rotated** object using a graphics library. This program should demonstrate the concept of **composite transformations** and the application of **2-D rotation** in computer graphics.

CO Mapped	CO 5
Pre -requisite	• Fundamentals of Computer Graphics • Coordinate System and Cartesian Plane • 2-D Geometric Transformations • Translation and Rotation Transformations • Matrix Multiplication • Homogeneous Coordinates • Basic programming skills of C++ • 64-bit Open source Linux • Open Source C++ Programming tool like G++/GCC
Learning Objective	• Understand the concept of **2-D rotation** in computer graphics. • Explain the need for **rotation about an arbitrary point** using composite transformations. • Implement the rotation of a 2-D object about an arbitrary point using C++. • Apply **translation** and **rotation transformation matrices** to achieve rotation about a specified point. • Compute the transformed coordinates of an object after rotation. • Display and compare the **original** and **rotated** objects using a graphics library. • Understand the role of **homogeneous coordinates** and **matrix multiplication** in implementing composite transformations. • Apply rotation transformations in real-world applications such as **computer-aided design (CAD), animation, robotics, game development, image processing, and engineering graphics**.

Theory:

Geometric transformations are essential operations in **Computer Graphics** that modify the position, orientation, size, or shape of graphical objects. Among these transformations, **rotation** is one of the most frequently used operations. Rotation changes the orientation of an object by turning it through a specified angle while preserving its shape and size.

In many practical applications, objects need to be rotated about a point other than the origin. Such a point is called an **arbitrary point** or **fixed point**. Rotation about an arbitrary point cannot be achieved using a single rotation matrix. Instead, it requires a **composite transformation**, which combines translation and rotation operations.

Applications of arbitrary point rotation include computer-aided design (CAD), robotics, animation, gaming, image processing, engineering graphics, and simulation systems.

2-D Rotation

Rotation is the process of turning an object through a specified angle in a two-dimensional plane.

If an object is rotated about the **origin**, the transformed coordinates are:

$$x' = x\cos\theta - y\sin\theta$$

$$y' = x\sin\theta + y\cos\theta$$

where:

- x,y = Original coordinates
- x′,,y′ = Rotated coordinates

Positive angles represent **anticlockwise rotation**, while negative angles represent **clockwise rotation**.

Rotation Matrix

The standard rotation matrix about the origin is:

$$\cos\theta \sin\theta 0 - \sin\theta \cos\theta 0\ 0\ 1$$

This matrix rotates the object only about the origin.

Rotation About an Arbitrary Point

Suppose the arbitrary point is

$$P(h,k)$$

The object cannot be rotated directly about this point.

Instead, the following sequence of transformations is applied:

Step 1: Translate the object

Move the arbitrary point to the origin

$$T(-h,-k)$$

Step 2: Rotate the object

Rotate the translated object by angle θ.

$$R(\theta)$$

Step 3: Translate back

Move the object back to its original position.

$$T(h,k)$$

Thus, the composite transformation is

$$T(h,k)\times R(\theta)\times T(-h,-k)T(h,k)$$

Composite Transformation

A composite transformation is obtained by combining two or more basic transformations.

For arbitrary point rotation, the sequence is:

1. Translation
2. Rotation
3. Reverse Translation

The order of transformations is important because matrix multiplication is **not commutative**.

Homogeneous Coordinates

To simplify multiple transformations, every point is represented using homogeneous coordinates.

Instead of

$$(x,y)$$

the point is represented as

$$(x,y,1)$$

Homogeneous coordinates allow all transformations to be represented using **3 × 3 matrices**, making matrix multiplication straightforward.

Advantages of Rotation about an Arbitrary Point

- Allows realistic movement of graphical objects.
- Essential for composite transformations.
- Preserves object size and shape.
- Supports animation and simulation.
- Provides flexibility in graphical modeling.

Applications

Rotation about an arbitrary point is widely used in:

- Computer-Aided Design (CAD)
- Robotics
- Animation
- Computer Games
- Engineering Drawing
- GIS Applications
- Image Processing
- Virtual Reality
- Mechanical Simulation

- Graphical User Interface (GUI) Design

Algorithm:

Algorithm for Rotation about an Arbitrary Point

1. Start.
2. Initialize the graphics window.
3. Accept the coordinates of the object (line, triangle, rectangle, or polygon).
4. Accept the coordinates of the arbitrary point (h,k)(h, k)(h,k).
5. Accept the angle of rotation θ.
6. Display the original object.
7. Translate the object so that the arbitrary point coincides with the origin:

- $x = x - h$
- $y = y - k$

8. Rotate each vertex using the rotation equations:

$$x' = x\cos\theta - y\sin$$

$$y' = x\sin\theta + y\cos$$

9. Translate the rotated object back:

$$x = x' + h$$

$$y = y' + k$$

10. Display the rotated object.
11. Close the graphics window.
12. Stop.

Conclusion:

This experiment demonstrates the implementation of **rotation of a 2-D object about an arbitrary point** using translation and rotation transformation matrices. Since direct rotation about an arbitrary point is not possible using a single transformation matrix, composite transformations are employed to achieve the desired result. This technique is fundamental in computer graphics and has wide applications in CAD, animation, robotics, game development, image processing, and engineering design.

Questions:

1. What is a 2-D geometric transformation?
2. Define rotation in computer graphics.
3. Why is rotation about an arbitrary point considered a composite transformation?
4. What are homogeneous coordinates, and why are they used?